

TanStack Trust: A Base for Domain-Owned Software Guarantees

Subtitle: Separating domain meaning, reusable evidence mechanics, and consumer policy

Status: Draft RFC for feedback

Audience: TanStack maintainers and contributors building libraries, domain packages, and agent-facing developer tools

Abstract

Code generation is becoming cheap. Guarantee work is not.

People still define what software must do, decide which checks deserve authority, verify that those checks reached the behavior they claim to test, preserve failures, determine whether old results still apply, and decide what the available evidence permits. Most of that work is manual today. TanStack Trust is an architecture for automating more of it without letting an agent approve its own claims.

A guarantee in this RFC is a bounded domain claim with declared evidence, conditions, and limits. It is not automatically a mathematical proof, certification, promise of universal coverage, or permission to act.

The architecture separates three responsibilities. A domain package defines the guarantee, the rules and checks that may support it, the contexts where evidence applies, and the known omissions. TanStack Trust maintains the evidence graph and its history: observations, support routes, applicability, freshness, contradictions, repair, and explanation. Consumer policy decides permission, severity, fallback, and deployment action. Workflows and interfaces help people and agents perform this work; they do not acquire authority of their own.

For the foreseeable future, Trust will be built and shipped inside TanStack DB Endpoints. It is an internal architecture and evidence contract, not a separate product or package roadmap. Endpoints provides the first bounded implementation and the recurring examples in this document. Those examples illustrate the architecture; they do not define its full range.

The current worktree implements a finite local kernel, verified file storage, one Endpoints check, and CLI, LSP, and MCP adapters over one service. That is a useful vertical slice. It does not establish that the registered rule is sound, that the architecture spans other domains, that agents can use it effectively, or that software is now safe to ship.

Status vocabulary

Completeness is the engineering goal. The RFC therefore distinguishes the complete architecture from the evidence available today.

Label	Meaning
Implemented	Present in the inspected local prototype.
Bounded evidence	Observed or tested only inside the named finite domain.
Proposed contract	Part of the architecture but not yet a stable public API.
Normative guidance	A rule for authoring or evaluating a domain package; not proof that packages follow it.
Absent	Required by the product vision but not exposed by the current implementation.
Unmeasured	No evidence yet establishes the claimed range, usability, or outcome.
Unresolved	The design needs an answer, but this RFC does not choose one.
Unselected	Several policies are possible and the architecture intentionally chooses none.

These labels appear where they matter. One mechanism can be implemented while its public contract is proposed and its product outcome remains unmeasured.

1. Why Trust exists

An agent can produce a feature in minutes. The resulting application still depends on human judgments that are harder to generate:

- What behavior matters?
- Under which conditions must it hold?
- What observation would support or contradict it?
- Did the check exercise the production behavior or only a substitute?
- Which code, data, configuration, environment, method, or external input could make yesterday's evidence irrelevant?
- What must be replayed after a failure?
- Who may turn the result into permission?

Cheaper generation also increases the number of candidate implementations. Each candidate brings more claims to evaluate, checks to run, exceptions to understand, and repairs that must remain legible after the session that made them. The durable value of developer tooling therefore comes from shared implementations that make application behavior more reliable, efficient, and maintainable, and from the responsibility they safely remove while leaving the remaining obligations visible.

The product opportunity is not another score for whether an agent is trustworthy. It is a system that makes the grounds for relying on a specific piece of software explicit, inspectable, renewable, and repairable.

Raw capability and deployability are different. A system can generate working code without establishing its operating bounds. A useful promise must say what it covers. A control needs evidence that it detects the problem it claims to detect. That evidence needs maintenance as the software changes. These principles are useful whether the surrounding institution is a library team, a deployment organization, or an independent evaluator. TanStack Trust adopts the principles, not a certification body, insurance scheme, universal standard, or overarching trust product. The distinction is discussed in more detail in this AIUC interview, but the architecture here does not depend on knowing that system.

The central question is:

What would justify relying on this change, in this domain, under these conditions—and what would make that justification expire?

Consider a small Endpoints example that will recur through the RFC. A retained query and a mutation share a build artifact. Endpoints would like to skip the query's refetch when the mutation cannot affect its result. That is useful only as a bounded claim. It depends on an authority baseline, the absence of pending optimistic or repair work, complete effect bounds, and disjoint read and write sets. External writes remain outside the law. Even a supported claim does not decide whether the product should enable the optimization.

The architecture must keep that example's meaning, evidence, history, and policy separate without making people or agents reconstruct the separation on every run.

Status: The cheap-code and continuing-guarantee-work premise is the product motivation selected for this RFC. The amount of human work removed, the safety of the automation, and the resulting software outcomes are **unmeasured**.

2. The authority Trust has—and refuses

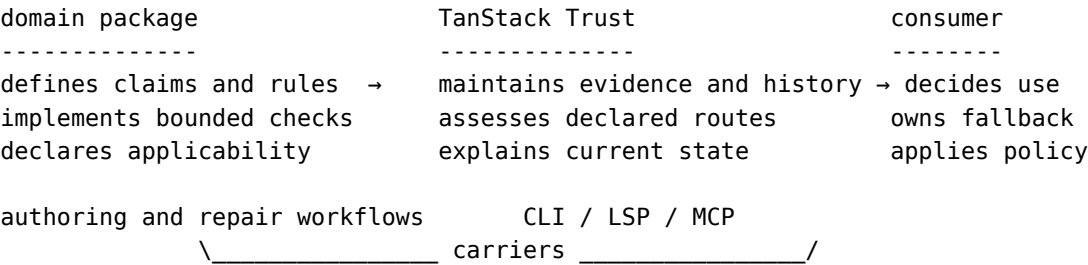
The practical question

Suppose an Endpoints check reports that the retained query may skip its refetch. Four statements can look deceptively similar:

- 1. Endpoints defines the conditions under which skipping is correct.
- 2. A check reports what it observed under those conditions.
- 3. Trust determines whether a declared support route is currently satisfied.
- 4. Product policy decides whether to skip the refetch.

No one statement has the authority of the other three.

The simple architecture



The system has three semantic owners and two carriers.

Part	Owns	Does not own
Domain package	Claim meaning, laws, check authority, evidence adequacy, applicability semantics, formal encodings, declared omissions	Generic transport or consumer permission
TanStack Trust	Evidence records, argument traversal, provenance, freshness, challenges, replay and repair history, shared operations	Domain truth, checker soundness, severity, or deployment action
Consumer policy	Permission, review requirements, severity, fallback, rollout, enforcement	Rewriting evidence state or erasing uncertainty
Workflow	Procedures for designing checks, maintaining evidence, and repairing failures	Granting authority because the steps were followed
Interface	Transporting the same operations through CLI, LSP, MCP, or another adapter	Changing semantics because a result appears in an editor or agent tool

This is an authority architecture, not a package diagram. All five parts may live in one repository, release, process, and user-facing product. That is the current plan for Endpoints. Keeping their responsibilities distinct prevents a domain rule, evidence record, editor diagnostic, and runtime decision from authorizing one another.

The same refusal applies to agents. An agent may propose a claim, a check, a rule, an argument, or a repair. It must not gain authority merely by producing a correctly shaped payload. Rule and checker admission needs an external, authorized boundary. A trusted origin is provenance, not proof of an unmeasured claim. Completing a workflow or closing a task is progress, not evidence.

“Small” describes the authority Trust may exercise. It does not predict the line count, storage footprint, or operational complexity required to preserve evidence well.

Status: The authority separation is a **proposed contract**. The local prototype implements much of the Trust responsibility and keeps consumer policy outside the kernel. It does not yet expose the stable internal extension boundary or admission governance the complete Endpoints product needs. Clean seams do not imply a current plan for an independent @tanstack/trust package.

3. The evidence graph

Why a graph is necessary

A green result is too small to explain why a claim may be relied on.

Imagine that a claim can be supported in either of two ways:

route A: claim C requires P1 AND P2

route B: claim C requires P3

Strong evidence for P1 cannot replace P2. P3 is an alternative to the entire first route, not another item in one flat checklist. A useful system must retain that shape.

The evidence objects

TanStack Trust uses six candidate primitives.

Object	Job
Claim	A versioned predicate about a subject under explicit conditions. It may exist before evidence.
Observation	An immutable reported result tied to one execution, method, origin, captured case, and dependency context.
Rule	A registered, versioned domain procedure declaring the premises and side conditions it accepts.
Argument	A proposed application of a rule to observations or premise arguments for a claim.
Applicability condition	The domain relation between captured evidence context and a requested use.
Challenge	A targeted objection or observed counterexample with unresolved and resolution history.

A run groups provenance for observations produced together. A task projects a missing premise or other gap into work that a later agent can discover. Neither is another evidence primitive or source of authority.

One run may report several observations. Agreement, work performed, latency, and response value are different laws even when recorded together. Each observation keeps its own claim, outcome, producer, method, captured case, and dependency context. A trusted origin or green aggregate does not grant a claim the run did not measure.

In the Endpoints miniature, the domain claim concerns one query and mutation. The check produces an observation about their bounded effect relation. The registered rule says which current observation it may admit. An argument cites that observation for the claim. The captured artifact, analyzer, inputs, and authority state determine applicability. A failing observation opens a challenge.

Support retains alternatives and joint work

Within an argument route, premises are conjunctive. Across arguments for the same claim, routes are alternatives. A flat list of gaps may be useful for search and task projection, but it is only a deduplicated inventory. It is not a completion plan.

Cycles do not create grounding. If A is supported only by B and B only by A, both remain unresolved until a route reaches independently admitted evidence. The current finite kernel implements this rule and covers it with bounded tests.

Illustrative TanStack Router case—not implementation evidence: Router automatically generates a file route tree with IDs, paths, and parent relationships. A domain package might claim that these generated relations are the ones the runtime uses to match URLs for a declared set of file-routing forms. One route could require both a generator observation and a runtime matching observation; another might use a separately authorized model check. The routes are alternatives, while the two observations in the first route remain joint work.

This is a hypothetical Trust application built on a concrete Router surface. The route-tree documentation describes the tree as the structure used to match a URL to a component tree, and the file-based quick start imports an automatically generated `routeTree.gen` into `createRouter`. The example does not establish that Trust already supports Router.

Different evidence forms retain different limits

The base can carry several evidence forms. They are different access methods, not a universal strength ladder.

Form	What the domain must retain	What it cannot claim automatically
Deterministic certificate	Analyzed semantics, certificate contents, verifier, unsupported constructs, trusted components	Completeness or soundness of the analyzer and verifier
Empirical or oracle check	Generated histories, independent expected result, exercised production boundary, observations, seeds or schedules, replay and reduction	Behavior outside the campaign or unobserved environment
Rubric-assessed inspection	Exact source passages and representation, rubric, assessor provenance, conclusions, limits	Deterministic proof or independence merely because review occurred
Formal model or solver check	Model or encoding, property, assumptions, scope or bound, tool configuration, witness, model-to-code claim	Truth of the encoding or correspondence with the implementation

Datalog can expose which facts and rules derive a conclusion. An SMT solver can return a satisfying model, an unsatisfiable core, or unknown. TLA+ and Alloy can return counterexample traces within a declared model or bound. Provenance expressions can preserve alternative and joint derivations.

Those outputs may become typed observations or explanations. The domain still owns the translation from software to facts, constraints, states, properties, and scopes. A solver’s success does not prove that the encoding described production. Shared inputs do not become independent votes, and coefficients do not become confidence.

Status: Claims, observations, rules, arguments, and challenges exist in the local kernel. Applicability is narrower than the proposed contract. Generic evidence ingestion, formal-check integration, task projection, and a stable public explanation schema are **absent** or **proposed**.

4. Current evidence: reach, applicability, and freshness

Four questions, not one verdict

When a check runs, the system must keep four kinds of state separate:

Question	Example states	Owner
What does the evidence establish?	Supported, contradicted, unresolved; no observation versus an inconclusive observation	Domain rules assessed by Trust
Did execution reach the promised behavior?	Reached production checkpoint, did not run, threw, reached only a substitute path	Check runner and domain contract
Did the operation itself fail?	Transport, process, timeout, malformed result, or other operational failure	Operational system; durable Trust integration unresolved
What may the product do?	Permit, deny, require review, choose fallback	Consumer policy

A check execution can succeed while returning fail because it correctly observed an application violation. A solver can successfully return unknown. A process can throw before observing the application. A consumer can deny an optimization even when its evidence route is supported. These are different facts.

If a check throws or misses its declared production checkpoint, it creates no correctness observation. CI or operational logs may retain the failure, but the current Trust model does not specify a durable agent-facing home for that history. That is an **unresolved** integration requirement, not a reason to fabricate an unresolved evidence observation.

In the Endpoints miniature, the observation checkpoint is the extracted production helper. Reaching only a fixture calculation would not establish that the production decision path ran.

Applicability belongs to the claim’s meaning

Evidence gathered under one context is useful only when the domain can state how that context relates to a requested use. Exact byte identity can be a conservative boundary. It is not a general account of semantic equivalence.

Two design questions remain open:

- Does the domain own the entire applicability matcher, or does Trust own a structural layer and call domain hooks for semantic comparison?
- Is applicability boolean, or must it distinguish applicable, inapplicable, and unknown?

The current kernel uses a private boolean comparison over explicitly captured dependency revisions. The proposed architecture treats applicability as an explicit, inspectable relation and preserves unknown rather than forcing a comparison it cannot justify.

Illustrative TanStack Query case—not implementation evidence: Query’s persistence contract includes a timestamp and maximum-age window, buster, hydration options, storage representation, and cache settings such as gcTime. A package might define when evidence about restoring a persisted client applies across those inputs. Trust could retain the dependencies and the package’s applicability result; it would not decide what counts as equivalent cache behavior.

Freshness is historical

Dependencies are named as code, data, configuration, environment, method, or external inputs. Every fingerprint change advances a monotonic revision.

```
analyzer revision 7: fingerprint a
analyzer revision 8: fingerprint b
analyzer revision 9: fingerprint a
```

Evidence captured at revision 7 does not revive at revision 9. The bytes match, but the old execution did not happen again in the current history.

The current rebuild invalidates only observations that captured a dependency whose revision changed. That is more precise than a global epoch. It is still only as complete as dependency discovery. Recorded context shows what was captured, not that every relevant input was captured.

For Endpoints, analyzer code, the shared artifact, pre-analyzed query and mutation summaries, authority state, environment, and method can all affect the safe-skip observation. A caller that omits one of those inputs may retain an apparently current result whose real applicability is unknown.

Status: Independent state axes, explicit dependency capture, monotonic revisions, and selective invalidation are **implemented** in the local rebuild. Complete dependency discovery and the public applicability contract remain **unresolved**.

5. Contradiction, causal repair, and expiry

A later pass is not automatically a repair

A failing observation opens a targeted challenge. A newer unrelated pass does not erase it. Resolution must link the original challenge to an applicable replay of the same law and case.

The order of events matters:

```
t1 failing observation is recorded; challenge opens
t2 trusted checker starts a fresh replay of the same case
t3 replay passes under applicable dependencies
t4 resolution links challenge to replay
```

The following do not satisfy that rule:

- a pass that started before t1 and returned later;
- another pass returned in the same failing batch;
- a cached result;
- a later green aggregate with no link to the case;
- a narrower claim that avoids the original obligation;
- a different strategy presented as if it repaired the original route; or
- a solver scope reset or new model with no model-to-case relation.

The checker must perform fresh measurements rather than return cached data. The Endpoints implementation enforces this conservative ordering for exact same-case replay.

Resolution is append-only history plus current applicability. If a dependency captured by the replay later changes, the replay's current authority can expire. The original challenge becomes relevant again, while the resolution record remains part of history. The system does not rewrite the past to claim that the repair never worked.

This is deliberately narrower than every useful notion of repair. A domain may eventually need alternate-strategy repair, cross-version mapping of a counterexample, narrower replacement claims, durable repair certificates, selective expiry, or distributed clocks. Those require explicit semantics. Trust cannot infer them from a later pass.

Status: Challenge creation, exact replay linkage, causal ordering, append-only history, and applicability-based expiry are **implemented in a bounded local form**. Richer identity and repair semantics remain **unresolved**.

6. Explanation is an operation

What must a later agent be able to see?

An assessment that returns unresolved has not done enough for an agent if it cannot say why.

Return to the two routes for claim C:

route A: C requires P1 AND P2

route B: C requires P3

If P1 exists and P2 and P3 are missing, the useful next work is not “collect two more checks.” One route needs P2; the other needs P3. Completing either route may support C. A flat gap list hides that choice.

A stable explanation should expose:

- the successful route or every unresolved route;
- jointly required premises and alternate ways to satisfy the claim;
- observations, rules, runs, and premise arguments on each path;
- the semantic dependencies captured by each observation;
- reach status and operational failures without turning them into findings;
- open challenge IDs and the evidence they defeat;
- applicability decisions and reasons for unknown;
- check bounds, omissions, witnesses, and counterexamples; and
- shared dependencies without converting their count into confidence.

This is not decorative prose around a verdict. It is a structured operation that lets a person or agent inspect the assessment, choose one useful route, and preserve the obligations another route still carries.

Tasks can be projected from these gaps. A task should identify the claim, route, missing premise, applicable context, and evidence form needed to make progress. Closing the task does not satisfy the premise. The resulting observation and argument must still pass the domain's rules.

Formal tools can enrich explanations without becoming the universal language of Trust. A Datalog proof tree may show producing rules and premises. An SMT model may serve as a witness, and an unsatisfiable core may show a sufficient conflict. A model checker may supply a trace. A provenance expression may make joint and alternate derivations visible. Trust can retain these structures as typed attachments while the domain owns their meaning and bounds.

In the Endpoints miniature, a complete explanation would show the exact safe-skip route, the check and producer version, the captured analyzer and artifact revisions, the authority baseline, any challenge opened by an overlapping effect, and the post-failure replay that resolved it. It would also say that external writers and analyzer completeness remain outside the observation.

Status: The kernel retains structured routes and challenges internally. The complete generic explanation is a **proposed contract** and not yet a stable public operation. Current adapter results expose only parts of it.

7. Safely automating the lifecycle

Automate work, not authority

The architecture becomes useful to agents when they can participate in the full lifecycle of a guarantee. Calling one fixed check is not enough. Agents need bounded ways to design checks, establish and maintain evidence, investigate failures, propose repairs, and discover remaining work.

They also need a hard boundary: participation does not let an agent authorize the rule that will judge its own output.

Design a check

A check-design workflow starts with one proposed claim, its consumer, the available source or code, and the consequence of a wrong conclusion. Before implementing the happy path, the author states:

- the law, subject, conditions, and bounded scope;
- a satisfying case, a violating case, and an unresolved case, plus who owns the fixtures that make those cases reachable;
- the reference or oracle and how it remains independent of production logic;
- the production path and exact observation checkpoint;
- the inputs and dependencies that can change applicability;
- trust assumptions, unsupported constructs, known omissions, and open decisions;
- the evidence route the check can satisfy; and
- the replay obligation after a known failure.

Matched boundary cases vary one premise at a time. Hostile cases attack false acceptance and unnecessary rejection. Reach controls prove that the expected production checkpoint was exercised. Fault controls mutate or otherwise disturb the implementation under test to show that the checker is sensitive to the fault it claims to detect. Diagnostics must also distinguish a missing joint premise from an available alternate route; otherwise an agent can do valid work that does not move the selected route toward completion.

For the Endpoints safe-skip law, the small oracle enumerates finite read and write subsets and computes intersection without calling the production helper. The design also needs cases with overlapping effects, incomplete effects, missing authority, different artifacts, and a deliberately damaged production decision. Each case answers a different question. None establishes behavior outside the finite campaign.

Illustrative TanStack Table case—not implementation evidence: Table exposes client-side sorting and filtering row-model stages. Its manual modes instead treat caller-provided rows as already sorted or filtered. A domain package might define a bounded property for one stage. The check would need explicit data and option bounds, a reference relation, the real row-model checkpoint, unsupported value types, and counterexamples that survive reduction. Trust would maintain the resulting evidence; Table would own the property.

Establish and maintain evidence

Maintenance begins from structured routes, current gaps, and open challenges. An agent chooses a route and one missing premise. It states what a proposed check can establish, captures relevant dependencies, invokes the check, records the observation, proposes an argument, and reassesses the original claim.

Partial evidence remains partial. If route A needs P1 and P2, establishing P1 does not hide P2. A warning, review decision, permitted optimization, or closed task cannot be recycled as evidence.

When context changes, the agent gathers new evidence only for uses affected by the changed dependencies. When a counterexample arrives, it enters repair rather than adding unrelated passing results.

Repair a claim or checker

Repair preserves the original law, inputs, context, observation, check, and rule versions. The agent first classifies the report:

- speculative objection;
- observed application bug;
- checker bug;
- generator or campaign gap; or
- operational failure.

It reproduces the failure, reduces the trace while retaining the same violation, repairs the correct layer, reruns the original case and broader controls, then links only the exact applicable replay as resolution.

A narrower claim keeps the original obligation visible. A different strategy gets its own argument. A new solver scope or model records new evidence; it does not become repair history without an explicit relation to the failed case.

The admission boundary

Agents should be able to propose claims, rules, checks, arguments, evidence, and repairs. They should also be able to inspect package versions, sources, omissions, and open gaps. An authorized review boundary must decide which rule and checker versions become trust roots for a domain package.

That boundary is not specified by the current prototype. Code review may be one part of it, but a complete product needs explicit ownership, versioning, capability, and revocation semantics.

Status: The three workflows are **implemented as documentation**. Generic agent operations and admission governance are **absent**. No agent study has established that the workflows reduce mistakes, improve diagnostics, shorten repair, or resist self-certification. Those outcomes are **unmeasured**.

8. One service, several interfaces, bounded persistence

Begin with the service, not the adapters

CLI, editor, and agent integrations should expose one operation service. The service owns shared operations and structured results; each adapter may present them differently for its user.

The stable service eventually needs operations to:

- discover domain packages and inspect versions, sources, rules, and omissions;
- propose and submit claims, observations, arguments, and challenges;
- execute authorized checks and ingest honestly labeled inspections;
- assess claims under an explicit requested context;
- explain support routes, gaps, challenges, dependencies, and witnesses;
- update dependency context and identify affected uses;
- propose and link exact repair replays;
- derive gap-based work without treating tasks as evidence; and
- negotiate schema and capability versions.

The operation result, not the adapter, defines the semantics. An LSP warning and MCP response derived from the same assessment are two paths to one result, not independent confirmation.

What the prototype exposes

Capability	CLI	LSP	MCP	Status
Initialize a verified store	init	evidence.request	Not advertised	Implemented
Summarize store and open challenges	status	evidence.request	evidence_status	Implemented
Assess one exact structured claim	assess	Document diagnostics and evidence.request	evidence_assess	Implemented
Run the Endpoints safe-skip check	run-endpoints	evidence.request	evidence_run_endpoints	Implemented, Endpoints-specific
Update dependency context	context	evidence.request	evidence_context	Implemented
Read observations and challenge history	history	evidence.request	evidence_history	Implemented
Link a challenge to a replay	resolve	evidence.request	evidence_resolve	Implemented
Publish supported, contradicted, or unresolved diagnostics	No	Yes	No	Implemented adapter behavior
Discover and inspect domain packages and rules	No	No	No	Absent
Author or register a versioned rule or check	No	No	No	Absent
Propose a generic argument with typed citations	No	No	No	Absent
Ingest generic observations or rubric inspections	No	No	No	Absent

Capability	CLI	LSP	MCP	Status
Request a stable, fully typed route explanation	Partial opaque assessment	Partial diagnostic data	Partial structured result	Proposed contract
Derive and manage gap-based agent work	No	No	No	Absent

The CLI accepts inline JSON, a referenced file, or standard input and returns a nonzero process status for top-level failure. The LSP reads JSON documents containing a claim, emits no diagnostic for support, an error for contradiction, and a warning for unresolved evidence. That severity mapping is current adapter behavior, not universal consumer policy. It also exposes the shared service through `evidence.request`. The MCP server advertises six tools and returns both text and structured results.

All three use the same local service and store. A shared integration test exercises those paths. No packaged editor configuration, contextual source diagnostic rendering, MCP installation, schema-quality pass, real agent registration, or cross-process deployment has been demonstrated.

What persistence protects

The current store validates counters and internal references, wraps state in a package-bound SHA-256 checksum, writes a temporary file followed by atomic rename, and serializes requests inside one service process.

These controls protect against partial writes and detect accidental or out-of-band modification when the actor cannot also rewrite the checksum. They do not provide signatures, authentication, a hostile-plugin boundary, multi-process transactions, or lost-update prevention between independent writers.

Status: The serialized local service, store, and three adapters are **implemented**. Distribution and actual agent use are **unmeasured**. Lifecycle operations, stable explanation schemas, coordinated writers, authentication, and a deployment security model are **absent** or **proposed**.

9. Domain integration and consumer action

What a domain package must supply

Trust can maintain evidence only after a domain makes its commitments explicit. A complete domain package declares:

1. the law, subject, conditions, and bounded scope of each claim;
2. evidence routes, including joint premises and alternatives;
3. the registered rule and checker versions authorized to interpret evidence;
4. the reference or oracle, production path, and observation checkpoint;
5. code, data, configuration, environment, method, and external inputs that can affect applicability;
6. unsupported constructs and known omissions;
7. reach and fault controls; and
8. the replay obligation for a known failure.

The package chooses the evidence method appropriate to its property. A certificate, empirical campaign, rubric inspection, solver model, or counterexample trace can all participate. The package owns the translation from its software to that evidence and the statement of what the method cannot establish.

Registered rule code remains a trust root. Trust can show that the rule was registered, its declared premises were traversed, and its observations are current under the available matcher. It cannot establish that the rule encoded the right law, an analyzer found every effect, a rubric was well chosen, or a formal model corresponds to production. Syntactic conformance does not become semantic authority.

In Endpoints, the CheckContract records the safe-skip law, source, domain, trusted analyzer components, production path, checkpoint, observations, omissions, reach witness, fault controls, and replay method. Its registered rule admits only a current skip observation from the exact check and producer.

Illustrative TanStack Form case—not implementation evidence: Form accepts Standard Schema validators and propagates schema errors to fields. Form-level validators can also return separate form and field errors. A domain package might define how a particular adapter maps a bounded class of validation results into those structures. The adapter version, schema library, configuration, input shape, and unsupported issue types could affect applicability. Trust could preserve observations and counterexamples; Form would define the correct mapping.

What consumer policy may decide

An assessment reports what the current registered routes establish. It does not decide whether an application should deploy a change, skip a refresh, require review, or tolerate missing evidence.

The separation prevents two opposite errors:

1. treating every unresolved claim as a universal veto; and
2. treating the absence of a counterexample as permission.

A project may require strict evidence for a destructive migration and permit review for a reversible local optimization. It may treat an observed, applicable contradiction more strongly than missing evidence. Those choices depend on stakes, reversibility, operating environment, and organizational authority.

Fallback is also domain-specific. Full refetch may sound conservative in an Endpoints discussion, but it is not universally safe when effects remain in flight or external systems are unobserved. Trust should expose the evidence and reach state that policy needs. It should not build one fallback into the meaning of unresolved.

No universal severity, strictness, confidence aggregation, evidence-strength order, investigation priority, permission, or fallback is selected.

Status: One Endpoints-owned package instantiates a CheckContract and rule. Generic package manifests, authoring, admission, rubric integration, formal integration, and consumer enforcement are **absent** or **proposed**. Universal policy remains intentionally **unselected**.

10. What exists in Endpoints today

Endpoints is the incubation host for Trust and the only current production-shaped domain package. This section audits how much of the architecture the local vertical exercises. It does not repeat the Endpoints product design.

The package is endpoints/safe-skip-refetch@1. Its claim concerns one retained query and one mutation analyzed under a shared artifact. A refetch may be reported as safe to skip only when:

- the query and mutation belong to the same artifact;
- the collection has a confirmed authority baseline;
- no optimistic or repair obligation is pending;

- both effect bounds are complete; and
- the query’s read set is disjoint from the mutation’s write set.

External writes are outside the law.

Case	Domain verdict	Evidence observation	Outside the result
Complete disjoint bounds	skip	pass, capable of supporting the exact claim	Whether policy enables the optimization
Overlap or missing authority	refresh	fail, opening a challenge for the exact use	Whether another strategy satisfies a broader requirement
Incomplete effects or different artifacts	unresolved	unresolved, with no invented counterexample	Whether missing evidence warns, blocks, or creates other work

The fixture supplies pre-analyzed query and mutation summaries plus authority state. The extracted production helper computes the bounded verdict over those inputs. The CheckContract and rule provide the domain-to-Trust handoff described earlier.

An independent oracle enumerates small read and write subsets over three table names and computes intersection without calling the production helper. This tests the set-intersection decision kernel. It does not test whether the SQL analyzer found every table, whether PostgreSQL returned the right values, whether an external writer changed data, or whether skip is the right product policy.

An omitted external writer shows the formal boundary. A solver may prove that the declared read and write sets are disjoint. If the model omitted a writer that can affect the query, the closed-world result does not establish the open-world production claim. The solver did its job; the domain encoding and captured context were incomplete.

At the inspected snapshot, the vertical has **bounded evidence** from 21 local tests and eight mutation controls across the kernel and rebuild. The rebuild is runnable working-tree state, not a committed or released package. Analyzer range, real PostgreSQL behavior, consumer enforcement, external writes, distribution, and agent use remain outside the result.

Endpoints helped form the architecture. It is therefore constructive evidence, not an independent test that Trust spans other domains. That limitation does not create a requirement to build another implementation. Trust can remain part of Endpoints while clean ownership, module, test, status, and interface seams preserve the option to extract it later.

11. Open decisions and build dependencies

The architecture integrates mechanisms from several systems. Each mechanism arrives with a boundary.

The evidence behind these transfers is not uniform. The list combines conceptual analogies, source-guided design donors, direct workflow lineage, unevaluated design transfers, supplied-source arguments, and bounded Endpoints implementation evidence. Concretely, proof assistants are a conceptual analogy; ATMS, Carneades, and the formal-system families are source-guided donors; Field Lab is direct workflow lineage; shadcn and Beads are unevaluated design transfers; bounded-control assurance is a source-guided product argument; and Endpoints alone supplies bounded implementation evidence. None is independent validation of Trust.

- **Proof assistants:** separating a small checking base from domain rules and proof construction is useful. Registered JavaScript rules remain trusted, and the prototype is not a theorem prover or a Lean-like trusted kernel.

- **ATMS and structured argument systems such as Carneades:** alternate support, joint premises, and defeaters inform the evidence graph. Static argument structure does not supply current applicability, execution reach, or repair causality.
- **Datalog systems such as Soufflé:** facts, recursive rules, and proof trees clarify derivation. Entailment remains relative to supplied facts and rules; positive derivation does not itself model applicability, defeat, expiry, or repair.
- **SAT and SMT solvers such as Z3:** models, unsatisfiable cores, and explicit unknown outcomes supply useful evidence forms. The encoding remains a trust root, cores need not be minimal, and solver state is not durable evidence history.
- **TLA+ and Alloy:** properties and counterexample traces retain explicit models and bounds. A model property is not automatically an implementation property, and absence of a bounded counterexample is not an unbounded guarantee.
- **Provenance semirings:** addition and multiplication express alternate and joint derivations. Positive provenance has no semantics-free account of negation or difference, and coefficients are not confidence or independent votes.
- **Evidence-oriented workflows, including Field Lab:** separating design, maintenance, hostile checks, repair, provenance, and human checkpoints clarifies work. Following a workflow does not validate the result.
- **shadcn-style lint diagnostics:** contextual findings, editable targets, and correction loops can make evidence usable near code. A plausible message is not proof that an agent can repair the problem or that the diagnostic is correct.
- **Beads-style task systems:** gaps and dependencies can become discoverable work for later agents. Task closure and counts do not grant support and can hide partial routes.
- **Bounded-control assurance:** separating capability from deployment, limiting promises, testing controls, and maintaining their evidence informs the product problem. Certification, insurance, independent-auditor institutions, and universal trust-product authority do not transfer.
- **Endpoints:** one package exercises claims, checks, context, contradiction, replay, storage, and three interfaces. It does not establish analyzer soundness, consumer policy, or generality.

No donor justifies putting one generic solver in Trust. The reusable architecture is the evidence lifecycle and its authority boundaries.

Authority and semantics

- Decide whether applicability is entirely domain-owned or split between a structural Trust layer and domain hooks.
- Decide whether applicability is boolean or applicable, inapplicable, and unknown.
- Define semantic equivalence, scope subsumption, and cross-version identity beyond exact canonical JSON.
- Decide whether speculative objections are distinct from challenges opened by observed failures.
- Define alternate-strategy repair, narrower replacement claims, durable repair certificates, selective expiry, and distributed execution clocks.
- Keep confidence aggregation, evidence-strength ordering, and investigation priority unselected until a domain or consumer supplies semantics.

Evidence and domain integration

- Define admission and review for checkers, analyzers, rubrics, models, and solver encodings.
- Improve semantic-dependency discovery and treatment of hidden inputs.
- Preserve model-to-code correspondence and formal-check bounds.
- Define a machine-readable failure object with environment, schedule, expected and actual results, original and reduced traces, reproduction class, and cleanup diagnostics.

Explanation, workflows, and interfaces

- Define a stable internal package manifest and extension contract.
- Add generic registration, argument proposal, check execution, observation ingestion, source inspection, explanation, gap, and dependent-use operations.
- Let agents propose rules without authorizing their own trust roots.
- Package LSP and MCP clients, improve schemas, negotiate capabilities, and exercise real agents.
- Specify where operational failures live so later agents can discover them without turning them into correctness observations.
- Preserve the boundary between evidence diagnostics and consumer action.

Runtime, security, and scale

- Choose a single-daemon or coordinated multi-process storage boundary.
- Define transactions between dependency updates and check reach.
- Add signatures, authentication, remote authority, hostile-process isolation including a hostile-plugin boundary, and evidence stores controlled by a separate actor only under an explicit threat model.
- Measure large argument graphs, long histories, recursive explanations, and long-lived projects.

Validation and product outcomes

- Broaden SQL-analyzer and PostgreSQL evidence for Endpoints.
- Keep independent range **unmeasured**; no second implementation is currently planned. If extraction is selected later, that decision may create the reason to test a second domain then; it is not a current milestone.
- Test agents designing checks, maintaining evidence, repairing failures, responding to diagnostics, and resisting self-certification.
- Use matched evaluations where fresh agents attempt the same task under a fixed budget, receive or do not receive a diagnostic, and then attempt a correction.
- Test human maintainer comprehension of the architecture and interfaces.
- Measure correctness, review burden, maintenance cost, correction time, and deployment outcomes.

Build by dependency

The complete Endpoints trust loop is the target vertical. It is not the first standalone implementation task.

1. Build the semantic core.

- Decide applicability ownership and result shape.
- Define typed claims, observations, rules, arguments, challenges, dependencies, and CheckContracts.
- Produce one complete explanation result.
- Establish authorized package and rule admission.

2. Complete the Endpoints agent loop.

- Add package discovery, check execution, argument proposal, evidence ingestion, gap, and dependent-use operations.
- Connect design, maintenance, and repair workflows to those operations.
- Connect current evidence and reach state to explicit Endpoints consumer policy without moving policy into Trust.
- Keep safe-skip bounded while expanding analyzer evidence and scoping dependencies by endpoint or artifact.

3. Ship and evaluate the real experience.

- Package CLI, LSP, and MCP integrations.

- Test actual editors and agents rather than adapter parity alone.
 - Measure the design, maintenance, diagnosis, and repair loop.
 - Harden multi-process coordination, authentication, hostile-process isolation, remote execution, and scale for the deployment Endpoints actually needs.
4. **Revisit extraction only when its value is concrete.**
- Another use, release boundary, ownership need, or repeated integration friction may justify a separate package.
 - Until then, explicit internal seams preserve the option without making extraction or a second implementation a milestone.

This sequence is a **proposed dependency order**, not frozen product policy. Another adapter or package boundary is not progress if Endpoints agents still cannot see and preserve the semantic obligations behind a result.

12. What a later agent must recover

TanStack Trust is complete enough to evaluate when a later agent can answer, without reconstructing the project from scratch:

- What bounded claim matters?
- Which routes could support it?
- Which premises are joint and which routes are alternatives?
- What was actually observed, by which check, at which production checkpoint?
- Which captured context makes the evidence applicable now?
- Which contradictions remain open?
- What exact post-failure replay repaired an earlier challenge?
- What evidence or repair authority has expired?
- What do the checks still not establish?
- Which work would close a real route gap?
- Who may admit the rule?
- Who has authority to decide what happens next?

That is the architecture for safely automating guarantee work. Domain packages own what their guarantees mean. Trust maintains the grounds, limits, and history. Workflows and interfaces let humans and agents participate. Consumer policy decides action. These are separate responsibilities inside one Endpoints product for now.

Completeness does not mean universal proof or permanent confidence. It means the system can represent what supports a domain claim, what defeats it, what makes the support current, how a repair was earned, and where the honest answer remains unresolved.